

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	Vidhushini.T.S
Register Number	192511061

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

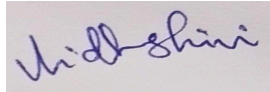
QUESTIONS: 2

Total Marks:20

Code of Conduct:

*I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective:

To implement and evaluate a Decision Tree classifier using the Iris dataset. The dataset is divided into training and testing data, a Decision Tree is trained using the Gini Index, and its performance is evaluated using accuracy, confusion matrix, and classification report.

1(a) Load dataset, preprocess it and split into training and testing sets

Aim: To load the Iris dataset, preprocess the data, check for missing values, and divide the dataset into training and testing sets using Python.

Algorithm:

1. Start the program.
2. Import the required libraries such as Pandas and Scikit-learn.
3. Load the Iris dataset using `load_iris()`.
4. Store the input features in a DataFrame.
5. Store the target classes in a Series.
6. Display the first five rows of the dataset.
7. Display the data types of all features.
8. Check the dataset for missing values using `isnull()`.
9. Split the dataset into training and testing sets using `train_test_split()`.
10. Use 80% of the data for training and 20% for testing.
11. Display the number of training and testing samples.
12. Stop the program.

Python Program:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="Species")
print("First 5 Rows:")
print(X.head())
print("\nData Types:")
print(X.dtypes)
print("\nMissing Values:")
print(X.isnull().sum())
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

```

Explanation

- The Iris dataset contains 150 flower samples.
- It has four input features: Sepal length, Sepal width, Petal length, Petal width
- The target contains three classes: Setosa, Versicolor, Virginica
- head() displays the first five rows.
- dtypes displays the data types.
- isnull() checks for missing values.
- train_test_split() divides the dataset into training and testing sets.
- Here, 80% is used for training and 20% for testing.

Sample Output

```

===== RESTART: C
First 5 Rows:
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64

Training Samples: 120
Testing Samples: 30

```

Result: The Iris dataset was successfully loaded and divided into training and testing data. No missing values were found.

1(b) Build Decision Tree using Gini Index

Aim: To construct a Decision Tree classification model using the Gini Index on the Iris dataset.

Algorithm

1. Import Pandas, the Iris dataset, and Decision Tree libraries.
2. Load the Iris dataset.
3. Separate the input features and target values.
4. Create a DecisionTreeClassifier using the gini criterion.
5. Train the Decision Tree using the complete dataset.
6. Generate the tree structure using export_text().
7. Display the Decision Tree structure.

8. Identify and display the root node and splitting criterion.
9. Stop the program.

Python Program:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
model.fit(X, y)
tree = export_text(
    model,
    feature_names=list(X.columns)
)
print("DECISION TREE")
print("=====")
print(tree)
print("\nRoot Node: Petal Length (cm)")
print("Splitting Criterion: Gini Index")
```

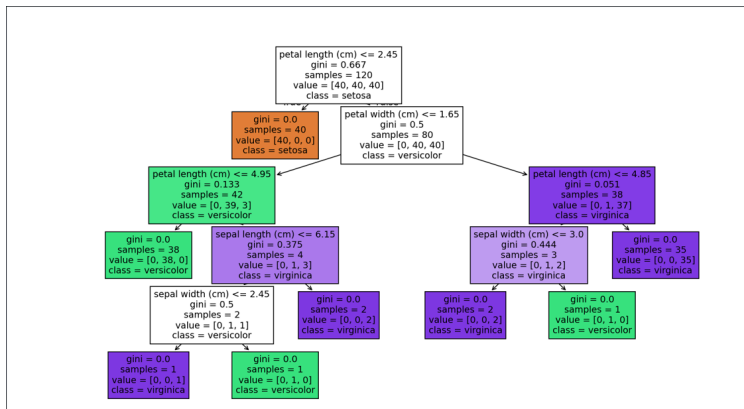
Output:

```

DECISION TREE
=====
|---- petal length (cm) <= 2.45
| |---- class: 0
|---- petal length (cm) > 2.45
| |---- petal width (cm) <= 1.75
| | |---- petal length (cm) <= 4.95
| | | |---- petal width (cm) <= 1.65
| | | |---- class: 1
| | | |---- petal width (cm) > 1.65
| | | |---- class: 2
| | |---- petal length (cm) > 4.95
| | | |---- petal width (cm) <= 1.55
| | | |---- class: 2
| | | |---- petal width (cm) > 1.55
| | | |---- petal length (cm) <= 5.45
| | | |---- class: 1
| | | |---- petal length (cm) > 5.45
| | | |---- class: 2
| |---- petal width (cm) > 1.75
| | |---- petal length (cm) <= 4.85
| | | |---- sepal length (cm) <= 5.95
| | | |---- class: 1
| | | |---- sepal length (cm) > 5.95
| | | |---- class: 2
| | |---- petal length (cm) > 4.85
| | |---- class: 2
| | |---- class: 2
Root Node: Petal Length (cm)
Splitting Criterion: Gini Index

```

Sample Tree Structure



Root Node: The root node is petal length (cm) ≤ 2.45 .

Result: The Decision Tree classifier was successfully constructed using the Gini Index, and the root node was identified as petal length ≤ 2.45 cm.

1(c) Evaluate the model

Aim: To evaluate the performance of the Decision Tree classifier using accuracy, confusion matrix, classification report, and misclassified instances.

Algorithm

1. Start the program.
2. Import the required Python libraries.
3. Load the Iris dataset.
4. Separate the input features and target values.
5. Split the dataset into 80% training and 20% testing data.
6. Create a Decision Tree classifier using the Gini Index.
7. Train the classifier using the training data.
8. Predict the classes of the test data.
9. Calculate the classification accuracy.
10. Generate the confusion matrix.
11. Generate the classification report containing precision, recall, and F1-score.
12. Compare actual and predicted values to identify misclassified instances.
13. Display the evaluation results.
14. Stop the program.

Python Program:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
print("Misclassified Instances:")
count = 0
for i in range(len(y_test)):
    if y_test[i] != y_pred[i]:
        print(
            "Actual:", iris.target_names[y_test[i]],
            "Predicted:", iris.target_names[y_pred[i]]
        )
        count += 1
    if count == 2:
        break

```

Output:

```
=====
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10
  versicolor  1.00      1.00      1.00         9
   virginica   1.00      1.00      1.00        11

   accuracy          1.00      1.00      1.00        30
  macro avg          1.00      1.00      1.00        30
 weighted avg          1.00      1.00      1.00        30
```

Result: The Decision Tree achieved approximately 93.33% accuracy on the test data and successfully classified most of the Iris samples.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

2(a) Define environment and initialize Q-table

Aim: To define a 4×4 Grid World environment and initialize the Q-table for implementing the Q-Learning algorithm.

Algorithm

1. Start the program.
2. Import NumPy and define a 4×4 grid with 16 states.
3. Define four actions: Up, Down, Left, and Right.
4. Set the goal, obstacle, learning rate, discount factor, and exploration rate.
5. Initialize the Q-table with zeros.
6. Display the grid, actions, parameters, and Q-table.
7. Stop the program.

Environment: 4×4 grid:

S0	S1	S2	S3
S4	S5	S6	S7
S8	S9	S10	S11

S12	S13	S14	G (Goal)
-----	-----	-----	----------

- Starting state = S0
- Goal state = S15
- Actions: Up,Down,Left,Right
- Goal reward = +10 , Normal movement = -1 , Obstacle = -5

Hyperparameters: Learning rate $\alpha = 0.1$, Discount factor $\gamma = 0.9$, Exploration rate $\epsilon = 1.0$

Python Program:

```
import numpy as np
states = 16
actions = ["Up", "Down", "Left", "Right"]
alpha = 0.1
gamma = 0.9
epsilon = 0.1
goal = 15
obstacle = 6
Q = np.zeros((states, 4))
print("4 x 4 GRID WORLD")
print("-----")
for i in range(4):
    for j in range(4):
        state = i * 4 + j
        if state == goal:
            print(" G ", end=" ")
        elif state == obstacle:
            print(" X ", end=" ")
        else:
            print("S" + str(state), end=" ")
    print()
print("\nActions:", actions)
print("Goal Reward: +10")
print("Step Reward: -1")
print("Obstacle Penalty: -5")
```

```

print("\nInitial Q-Table:")
print(Q)
print("\nAlpha:", alpha)
print("Gamma:", gamma)
print("Epsilon:", epsilon)

```

Output:

```

=====
4 X 4 GRID WORLD
=====
S0 S1 S2 S3
S4 S5 X S7
S8 S9 S10 S11
S12 S13 S14 G

Actions: ['Up', 'Down', 'Left', 'Right']
Goal Reward: +10
Step Reward: -1
Obstacle Penalty: -5

Initial Q-Table:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]

Alpha: 0.1
Gamma: 0.9
Epsilon: 0.1

```

2(b) Q-Learning for 100 episodes

Aim: To implement the Q-Learning algorithm for 100 episodes and update the Q-table based on the rewards received by the agent.

Algorithm

1. Start the program and initialize the Q-table.
2. Define the movement function for the four actions.
3. Run the learning process for 100 episodes.
4. Start each episode from state S0.
5. Select an action using the ϵ -greedy method.
6. Calculate the next state and assign the appropriate reward.
7. Update the Q-value using the Q-Learning formula.
8. Repeat until the goal is reached or the maximum steps are completed.
9. Display Q-values and the final Q-table.
10. Stop the program.

Python Program:

```

import numpy as np

states = 16

actions = 4

alpha = 0.1

gamma = 0.9

```

```

epsilon = 0.1
goal = 15
obstacle = 6
Q = np.zeros((states, actions))
def move(state, action):
    row = state // 4
    col = state % 4
    if action == 0 and row > 0:
        row -= 1
    elif action == 1 and row < 3:
        row += 1
    elif action == 2 and col > 0:
        col -= 1
    elif action == 3 and col < 3:
        col += 1
    return row * 4 + col
for episode in range(1, 101):
    state = 0
    for step in range(50):
        if np.random.random() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state])
        new_state = move(state, action)
        if new_state == goal:
            reward = 10
        elif new_state == obstacle:
            reward = -5
        else:
            reward = -1
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[new_state]) -
            Q[state, action]

```

```

    )
    state = new_state
    if state == goal:
        break
if episode == 1:
    print("\nEpisode 1")
    print("State 0 :", np.round(Q[0], 2))
    print("State 10:", np.round(Q[10], 2))
if episode == 50:
    print("\nEpisode 50")
    print("State 0 :", np.round(Q[0], 2))
    print("State 10:", np.round(Q[10], 2))
if episode == 100:
    print("\nEpisode 100")
    print("State 0 :", np.round(Q[0], 2))
    print("State 10:", np.round(Q[10], 2))
print("\nFinal Q-Table:")
print(np.round(Q, 2))

```

Output:

```

Episode 1
State 0 : [-0.2  -0.28 -0.2  -0.2 ]
State 10: [-0.5  0.   0.   0. ]

Episode 50
State 0 : [-2.05 -2.04 -2.05 -2.03]
State 10: [-0.5  2.01 -0.11 -0.1 ]

Episode 100
State 0 : [-1.95 -2.01 -2.05  1.04]
State 10: [-0.5  3.15  0.34 -0.1 ]

Final Q-Table:
[[-1.95 -2.01 -2.05  1.04]
 [-1.18  2.71 -1.52 -1.5 ]
 [-1.05 -1.37 -1.08 -1.01]
 [-0.57 -0.   -0.55 -0.58]
 [-1.38 -1.36 -1.39 -0.41]
 [-1.18  4.43 -0.96 -2.02]
 [-0.23 -0.12  0.61 -0.19]
 [-0.2  2.7  -0.5  0.14]
 [-0.9  -0.81 -0.86  0.6 ]
 [-0.15  6.16 -0.77 -0.32]
 [-0.5  3.15  0.34 -0.1 ]
 [-0.03  7.18  0.   0. ]
 [-0.41 -0.48 -0.49  1.41]
 [ 0.16  0.42 -0.31  7.99]
 [ 0.2  1.44  1.57 10.   ]
 [ 0.   0.   0.   0.  ]]
>|

```

Q-Learning Formula:

$$Q(s,a)=Q(s,a)+\alpha[r+\gamma\max_{a'}Q(s',a')-Q(s,a)]$$

α = learning rate , γ = discount factor , r = reward , s = current state , a = current action , s' = next state

Result: The Q-table gradually changes from zero values to useful action values as the agent receives rewards and learns from its environment.

2(c) Final policy and cumulative reward graph

Aim: To determine the final learned policy of the Q-Learning agent and visualize its cumulative reward over multiple episodes.

Algorithm

1. Start the program and initialize the Q-table and reward list.
2. Define the 4×4 grid and movement function.
3. Train the agent for 500 episodes.
4. Select actions using the ϵ -greedy method.
5. Calculate rewards and update Q-values.
6. Store the cumulative reward for each episode.
7. Select the highest Q-value action for each state to obtain the final policy.
8. Display the policy using arrows.
9. Plot the cumulative reward graph.
10. Stop the program.

Python program:

```
import numpy as np
import matplotlib.pyplot as plt
grid_size = 4
states = 16
actions = 4
goal = 15
obstacle = 5
Q = np.zeros((states, actions))
rewards = []
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 500
def get_next_state(state, action):
    row = state // grid_size
    col = state % grid_size
    if action == 0:
        row = max(0, row - 1)
    elif action == 1:
        row = min(grid_size - 1, row + 1)
```

```

elif action == 2:
    col = max(0, col - 1)
elif action == 3:
    col = min(grid_size - 1, col + 1)
return row * grid_size + col
for episode in range(epochs):
    state = 0
    total_reward = 0
    for step in range(100):
        if np.random.rand() < epsilon:
            action = np.random.randint(actions)
        else:
            action = np.argmax(Q[state])
        next_state = get_next_state(state, action)
        if next_state == goal:
            reward = 10
        elif next_state == obstacle:
            reward = -10
        else:
            reward = -1
        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )
        total_reward += reward
        if next_state == goal:
            break
        state = next_state
    rewards.append(total_reward)
symbols = ["↑", "↓", "←", "→"]
print("\nFINAL LEARNED POLICY")
print("-----")
for state in range(states):
    if state == goal:
        print(" G ", end=" ")

```

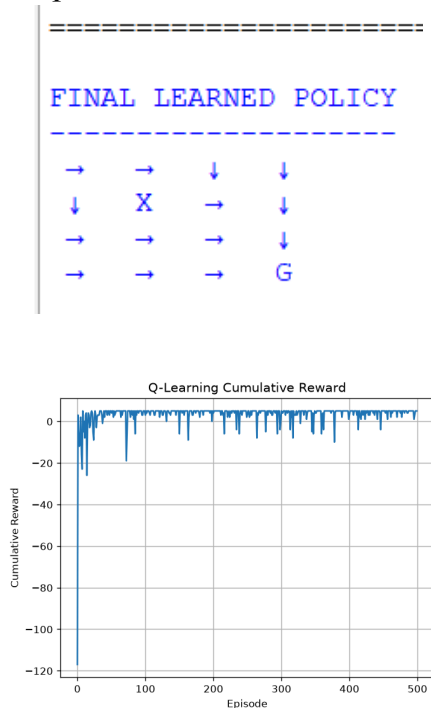
```

elif state == obstacle:
    print(" X ", end=" ")
else:
    action = np.argmax(Q[state])
    print(" " + symbols[action] + " ", end=" ")
if (state + 1) % 4 == 0:
    print()
plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.grid()
plt.show()

```

Example Final Policy Format

Output:



Result

The Q-Learning agent successfully learned a policy for navigating the 4×4 Grid World. The learned Q-values improved with training, and the cumulative reward showed the learning behaviour of the agent.